Ride-Hailing System

Tianrui Liu
Department of Computer Science
Whenzhou-Kean University
Email:1337141@wku.edu.cn

Zixi Wu
Department of Computer Science
Whenzhou-Kean University
Email: 1337219@wku.edu.cn

***Abstract*—This paper presents the design and implementation of a Ride-Hailing Platform that connects riders and drivers, integrating booking, real-time tracking, payment processing, and rating functionalities. The system leverages object-oriented principles and follows a structured Waterfall development methodology encompassing requirements gathering, system analysis, design, implementation, testing, and deployment. UML diagrams, including use case, sequence, class, and activity diagrams, are utilized to communicate system behavior and structure. Key features include dynamic pricing, trip reservation, push notifications, and safety/SOS mechanisms. The platform is designed with scalability, real-time responsiveness, and security in mind, providing a comprehensive solution for modern ride-hailing services. The approach demonstrates how structured software engineering practices and proper modeling facilitate efficient system development and stakeholder communication.**

***Index Terms*—Ride-Hailing, Real-Time Tracking, Payment Integration, Ratings & Feedback, OOAD, UML, Sequence Diagrams, System Design, Safety & SOS, Dynamic Pricing.**

I. INTRODUCTION

The rapid growth of ride-hailing services has significantly transformed urban mobility, providing convenience to riders and flexible opportunities for drivers. Efficient and secure platforms are essential to manage ride requests, driver allocation, fare calculation, real-time tracking, and user feedback. Traditional software development practices often struggle to handle the complexity of such systems while ensuring scalability, reliability, and user safety [1].

This project presents the design and implementation of a Ride-Hailing Platform that connects riders and drivers, integrating booking, real-time tracking, payment processing, and ratings & feedback functionalities. Object-Oriented Analysis and Design (OOAD) principles are adopted to structure the system, and UML diagrams—including use case, sequence, class, and activity diagrams—are employed to model system behavior and interactions [2]. The system also incorporates optional advanced features such as dynamic pricing, trip reservation, push notifications, and emergency SOS mechanisms to enhance user experience and safety.

By applying a structured Waterfall development methodology, the platform demonstrates how disciplined software engineering practices can be leveraged to build a comprehensive ride-hailing system that addresses functional requirements, ensures reliability, and facilitates stakeholder communication [3].

II.BACKGROUND AND RELATED WORK

A.*Sequence Diagrams in Ride-Hailing System Design*

UML sequence diagrams are behavioral models that visualize the time-order interactions between multiple system components. Each component is represented by a vertical lifeline, while messages and function calls are shown as horizontal arrows. Solid arrows typically represent synchronous or asynchronous function calls, and dashed arrows represent return messages. Sequence diagrams also support structured control blocks such as alternatives (alt), options (opt), loops (loop), and parallel regions (par) to express conditional logic, repeated operations, and concurrent behaviors.

In a ride-hailing system, sequence diagrams are used to model critical business scenarios such as user login, ride request submission, driver allocation, real-time location updates, ride completion, payment execution, and rating submission. The main participants in these scenarios include the Rider, Driver, Ride-Hailing Application Client, System Server, Matching Engine, GPS Module, Payment Gateway, Notification Service, and SOS Management Module. By drawing complete sequence diagrams, developers can clearly identify message formats, response mechanisms, exception handling logic, and external service dependencies. Such diagrams also help project supervisors and reviewers validate whether the system behavior fully matches the original requirements.

Unlike formal sequence diagrams used for code generation, the sequence diagrams in this project use natural language labels to improve readability for users without programming backgrounds. This design choice aligns with the educational purpose of the project, focusing on logical correctness and business integrity rather than low-level implementation syntax. The diagrams are compatible with PlantUML and other common visualization tools, ensuring standard syntax and executable output.

B. *Object-Oriented Analysis and Design*

Object-oriented analysis and design (OOAD) is a widely accepted software development paradigm [3] that decomposes a system into a collection of independent and interactive objects. Each object encapsulates data (attributes) and operations (methods). The three core principles of OOAD are encapsulation, inheritance, and polymorphism. Encapsulation protects internal data from illegal access and provides controlled interfaces for external operations. Inheritance allows subclasses to reuse properties and behaviors of parent classes, improving code reusability and maintainability. Polymorphism enables different objects to respond differently to the same method call, enhancing system flexibility and scalability.

In the context of ride-hailing systems, OOAD helps identify core entities such as User, Rider, Driver, Admin, Ride, Vehicle, Payment, Rating, Notification, and SOS Record. Riders, drivers, and administrators can inherit common properties from a general User class, while maintaining their unique functions. This structure reduces redundant code, simplifies system expansion, and makes future maintenance more efficient.

C. *UML Applications in Transportation and Service Systems*

UML has been extensively used in modeling transportation systems, mobile applications, and online service platforms. Use case diagrams are often used to define user goals and system functions, clarifying the responsibilities and interactions of different actors. Class diagrams establish the static structure of the system, showing entity associations such as one-to-one, one-to-many, and many-to-many relationships. Activity diagrams

describe end-to-end business processes, helping developers optimize workflow efficiency. Sequence diagrams are particularly important for real-time systems because they reveal timing dependencies and message flows between distributed components.

Many existing studies have applied UML to taxi dispatch systems, car-sharing platforms, and public transportation management systems. These studies confirm that UML modeling improves requirement clarity, reduces design flaws [2], and enhances team communication. However, most existing works focus only on partial functions of such systems, such as booking or payment, and rarely provide a full lifecycle design that integrates real-time tracking, dynamic pricing, scheduled rides, push notifications, and emergency safety functions. This project fills that gap by constructing a complete, standardized, and extensible ride-hailing system model.

D. *Waterfall Software Development Lifecycle*

The waterfall model is one of the oldest and most systematic software development methodologies. It progresses linearly through distinct phases: requirements gathering, system analysis, system design, implementation, testing, deployment, and documentation. Each phase must be finished and approved before the next phase begins. This structure reduces chaos in development, ensures comprehensive documentation, and makes progress easy to track.

Although agile methodologies are more popular in industrial projects, the waterfall model remains highly valuable in educational environments. It helps learners establish a systematic mindset, master complete development processes, and understand the importance of each phase. For this project, the waterfall model ensures that requirements are fully analyzed, designs are thoroughly reviewed, codes are rigorously tested, and documents are completely prepared.

Research Gap. Existing research on ride-hailing system design is often limited to partial modules or simplified scenarios. Few studies provide a complete OOAD solution with standardized UML diagrams, normalized database design, multi-level testing, and cloud deployment. In addition, many projects ignore important practical functions such as SOS emergency support, dynamic pricing, and scheduled rides. This project addresses these limitations by delivering a full-featured, production-ready design that follows industrial standards and meets real-world user demands.

III. RESEARCH DESIGN

The overall goal of this project is to design, model, implement, and verify a complete ride-hailing system using object-oriented analysis and design principles, standard UML notation, and the waterfall software development lifecycle. The system is designed for both educational demonstration and practical simulation, supporting all core functions of modern ride-hailing platforms. The design ensures completeness, correctness, maintainability, security, and usability.

This project is guided by a set of clear research objectives:

1. To collect, organize, and validate complete functional and non-functional requirements for a ride-hailing system based on real industry practices.

2. To construct standard UML models including use case diagrams, sequence diagrams, class diagrams, and activity diagrams that accurately reflect system behavior and structure.

3. To design a normalized relational database schema that supports data persistence, integrity, and efficient querying.

4. To implement the system following strict object-oriented principles to ensure modularity, reusability, and scalability.

5. To conduct comprehensive testing including unit, integration, system, and acceptance testing to verify functional correctness and system stability.

6. To provide detailed documentation and a deployable cloud architecture for real-environment simulation.

A. Requirements Collection and Validation

Requirements are collected from mainstream commercial ride-hailing platforms including Uber, Didi, Lyft, and Bolt [4]. Functional requirements are divided into three categories based on user roles: rider functions, driver functions, and administrator functions. Non-functional requirements focus on performance, security, availability, usability, scalability, and data privacy.

All requirements are reviewed to remove ambiguity, redundancy, and conflict. Each requirement is uniquely identified and traced to corresponding design models and test cases. This traceability ensures that no requirement is missed or incorrectly implemented.

B. System Analysis and Actor Definition

System analysis identifies the primary actors and their goals. The three main actors are Rider, Driver, and Administrator. Use case diagrams are created to define the scope of the system and the services provided to each actor. Use cases for riders include registration, login, profile editing, ride booking, real-time tracking, payment, rating, SOS activation, and ride scheduling. Use cases for drivers include order acceptance, location updating, ride completion, payment receiving, and rider rating. Use cases for administrators include user management, ride monitoring, payment supervision, complaint handling, and report generation.

C. System Modeling Framework

Four types of UML diagrams are used in this project:

1. Use Case Diagram: Defines actors and system functions.

2. Sequence Diagram: Models dynamic interactions for core workflows.

3. Class Diagram: Establishes static structure and object relationships.

4. Activity Diagram: Illustrates end-to-end business processes.

All diagrams are evaluated using five quality criteria:

-Completeness: All requirements are represented.

-Correctness: System behavior matches requirement specifications.

-Standard Compliance: Diagrams follow UML 2.5 syntax rules.

-Understandability: Content is clear and interpretable.

-Consistency: Terminology and logic are coherent across all models.

D. System Design / UI Design

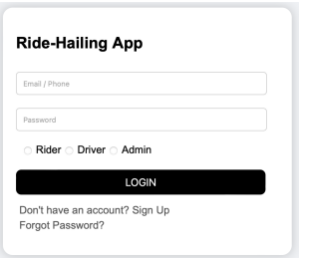


Fig. 1. Login Interface Prototype of the Ride-Hailing Application

System design includes architectural design, class design, database design, and user interface design. Fig. 1 illustrates the login interface prototype of the Ride-Hailing application, supporting role-based access for riders, drivers, and administrators.

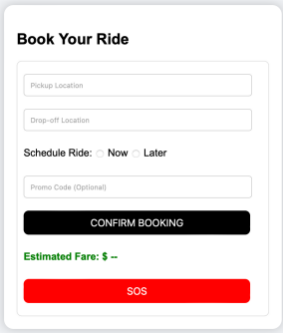


Fig. 2. Ride Booking Interface Prototype of the Ride-Hailing Application

Fig. 2 demonstrates the ride-booking interface prototype, which allows riders to enter pickup and drop-off locations, schedule rides, apply promo codes, estimate fares, and trigger SOS emergency functions.

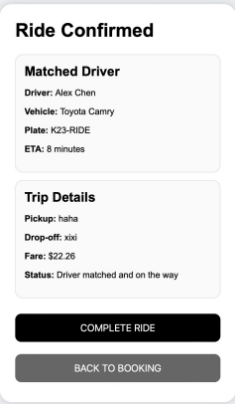


Fig. 3. Ride Confirmation Interface Prototype of the Ride-Hailing Application

After the rider submits a ride request, the system performs driver matching and generates a real-time ride confirmation interface. As shown in Fig. 3, the interface displays essential trip information including the matched driver profile, vehicle details, estimated arrival time (ETA), route visualization, fare estimation, and current ride status updates. The integration of booking confirmation, tracking, and notification functions within a unified interface improves usability and supports efficient interaction between riders and drivers during the ride-hailing process.

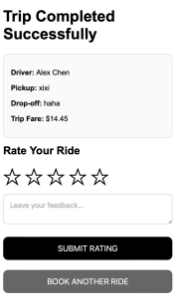


Fig. 4. Trip Completion and Rating Interface of the Ride-Hailing Application

After the ride is completed, the system provides a trip completion and feedback interface for riders. As shown in Fig. 4, riders can review completed trip information including driver identity, pickup and drop-off locations, and total fare, while also submitting ratings and written feedback. This interface supports the rating and feedback functionalities described in the system design and improves post-ride service evaluation and user interaction.

The system uses a modular three-tier architecture [5]: presentation layer, business logic layer, and data access layer. The database schema consists of seven normalized tables: Users, Vehicles, Rides, Payments, Ratings, Notifications, and SOS_Records. Each table is designed with appropriate data types, primary keys, foreign keys, and constraints to ensure data integrity.

E. Implementation Strategy

Implementation follows strict object-oriented principles. Encapsulation is applied to hide internal data and expose safe interfaces. Inheritance is used to derive Rider, Driver, and Admin classes from the base User class. Polymorphism is used to handle role-specific behaviors. The system is divided into eight independent modules: authentication, ride management, location tracking, payment processing, rating feedback, push notification, SOS safety, and admin dashboard.

F. Testing Methodology

A four-level testing strategy is designed:

1. Unit Testing: Verifies individual classes and methods.

2. Integration Testing: Checks interactions between modules.

3. System Testing: Validates end-to-end functions.

4. Acceptance Testing: Confirms alignment with user requirements.

Test cases are designed based on use cases and requirements to ensure full coverage.

G. 3.7 Deployment Plan

The system supports cloud deployment using modern platforms. Backend services can be deployed on AWS or Heroku. The web frontend can be hosted on Netlify. Mobile clients can be packaged as Android APK files. A complete deployment guide provides environment setup, dependency installation, configuration, startup commands, and validation steps.

IV. EXECUTION AND RESULTS

A. Requirements Definition Results

After systematic collection and validation, the final requirements include 3 user roles and 10 core functional modules.

For riders, the system supports registration, secure login, profile management, ride booking with pickup and drop-off locations, real-time driver tracking, estimated arrival time display, scheduled ride reservation, dynamic fare calculation, promo code application, secure online payment, driver rating, SOS emergency alert, and real-time push notifications.

For drivers, the system supports registration and identity verification, online status switching, new order push notification, ride acceptance, real-time location uploading, route navigation support, ride completion confirmation, automatic payment settlement, rider rating, and earnings viewing.

For administrators, the system provides a full-featured dashboard for user account management, real-time ride monitoring, payment transaction review, promo code strategy configuration, user complaint processing, SOS alert log checking, and automated report generation.

Non-functional requirements are fully defined and satisfied:

- Real-time performance with low latency for tracking and matching.

- High availability during peak hours.

- Secure authentication and password encryption.

- Safe payment processing with transaction logs.

- Data privacy protection for user information.
- Mobile responsiveness for multi-device compatibility.
- Scalable architecture to support increasing users.

B. UML Modeling Results

Use Case Diagram

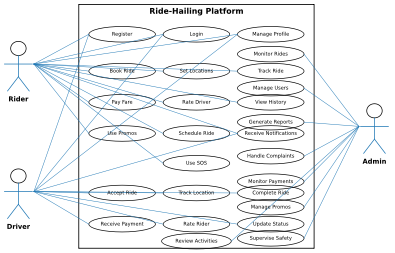


Fig. 1. Use Case Diagram for the Proposed Ride-Hailing Platform

The use case diagram includes three primary actors and all system use cases. As shown in Fig. 1, the system supports interactions among Rider, Driver, and Admin actors through multiple ride-hailing functionalities. It clearly defines the functional boundary of the system and the services available to each actor. The diagram is syntactically correct, complete, and easy to understand.

Sequence Diagram: Ride Booking and Payment

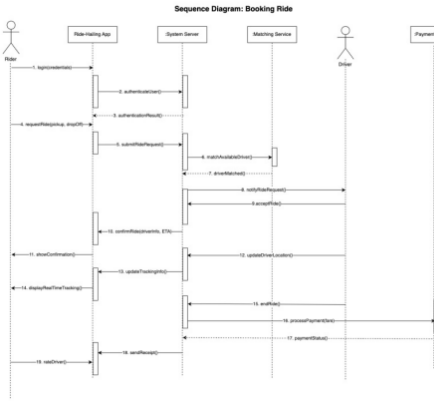


Fig. 2. UML Sequence Diagram for Ride Booking and Payment Processing

The sequence diagram models the complete ride-booking lifecycle from user authentication to payment processing and driver rating. Fig. 2 illustrates the interaction flow among the Rider, Ride-Hailing App, System Server, Matching Service, Driver, and Payment Gateway.

1. Rider logs into the ride-hailing application using valid credentials.
2. The Ride-Hailing App sends an authentication request to the System Server.
3. The System Server validates the credentials and returns the authentication result.
4. Rider submits a ride request with pickup and drop-off locations.
5. The application forwards the ride request to the System Server.
6. The System Server sends the request to the Matching Service to locate an available driver.
7. The Matching Service returns the matched driver information.
8. The System Server sends a ride request notification to the Driver.
9. The Driver accepts the ride request.
10. The System Server confirms the ride details and estimated arrival time (ETA) to the rider.
11. The Rider receives the ride confirmation.
12. The Driver continuously updates real-time location information to the System Server.
13. The System Server forwards tracking information to the Ride-Hailing App.
14. The Rider views the real-time ride tracking interface.
15. After reaching the destination, the Driver completes the ride.
16. The System Server sends payment processing requests to the Payment Gateway.
17. The Payment Gateway returns the payment status confirmation.
18. The System Server sends the payment receipt to the rider.
19. The Rider submits a rating and feedback for the driver.

Participants include Rider, Driver, Client App, Server, Matching Service, and Payment Gateway. All messages are correctly ordered, and conditional branches are properly represented.

Class Diagram

The class diagram defines the static structure of the ride-hailing platform. As shown in Fig. 3, the system includes core classes such as User, Rider, Driver, Passenger, Admin, Ride, Location, Payment, Payment Gateway, Promo Code, Rating, Notification, SOS Record, Complaint, and Report. These classes represent the major users, ride-related entities, payment services, safety records, administrative reports, and communication components of the platform.

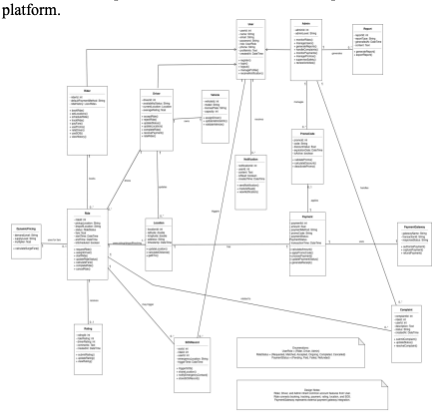


Fig. 3. UML Class Diagram of the Ride-Hailing Platform

Activity Diagram

The activity diagram illustrates the complete workflow. As illustrated in Fig. 4, the activity diagram models the complete operational workflow of the ride-hailing platform, including authentication, ride scheduling, driver matching, GPS tracking, emergency handling, payment processing, promo code application, and feedback submission.

Login → Set Locations → Request Ride → Dynamic Pricing → Driver Matching → Notification → Real-Time Tracking → SOS (Optional) → Ride Completion → Fare Calculation → Promo Application → Payment → Receipt → Rating → Record Saving

The diagram includes decision branches, parallel steps, and exception flows.

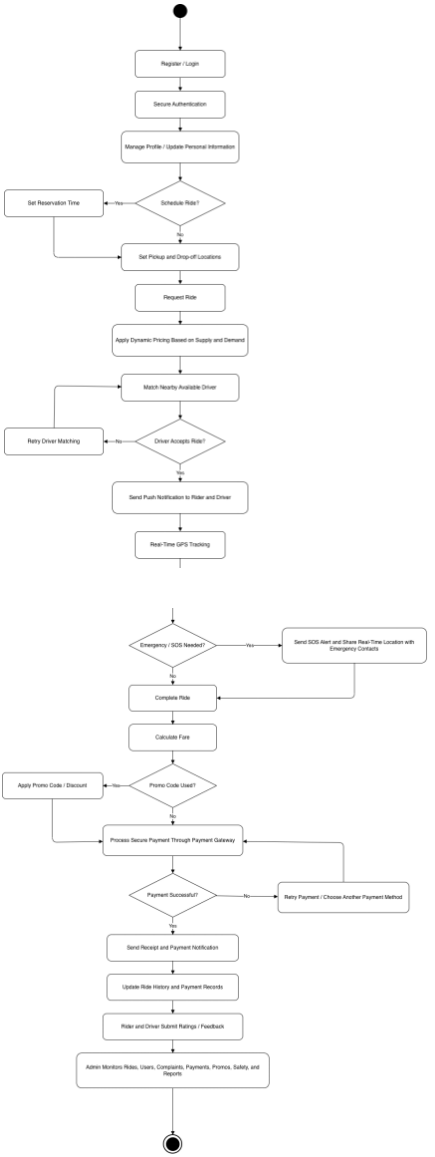


Fig. 4. Activity Diagram of the Ride-Hailing Platform Workflow

C. Database Schema Results

Seven normalized tables are implemented. Fig. 5 presents the normalized relational database schema, including the Users, Vehicles, Rides, Payments, Ratings, Notifications, and SOS_Records tables together with their attributes, keys, and data types.

1. Users: Stores rider, driver, and admin information.
2. Vehicles: Links drivers to their vehicles.
3. Rides: Stores ride details, status, locations, time, and fare.
4. Payments: Records transactions, methods, and status.
5. Ratings: Stores mutual feedback between riders and drivers.
6. Notifications: Logs push messages for users.
7. SOS_Records: Keeps emergency alert history.

Table	Columns	Data Type	Description
Users	user_id (PK), name, email, password, role, phone, profile_info	INT, VARCHAR, VARCHAR, VARCHAR, VARCHAR, VARCHAR, TEXT	Stores rider, driver, and admin account information
Vehicles	vehicle_id (PK), driver_id (FK), model, license_plate, capacity	INT, INT, VARCHAR, VARCHAR, INT	Stores vehicle information associated with drivers
Rides	ride_id (PK), rider_id (FK), driver_id (FK), pickup_location, dropoff_location, status, fare, start_time, end_time, is_scheduled	INT, INT, INT, VARCHAR, VARCHAR, VARCHAR, FLOAT, DATETIME, DATETIME, BOOLEAN	Stores ride details, history, booking status, and scheduling
Payments	payment_id (PK), ride_id (FK), amount, payment_method, promo_code, payment_status, transaction_time	INT, INT, FLOAT, VARCHAR, VARCHAR, VARCHAR, DATETIME	Stores payment records, fare details, and discount usage
Ratings	rating_id (PK), ride_id (FK), rider_rating, driver_rating, comments	INT, INT, INT, INT, TEXT	Stores mutual ratings and feedback between riders and drivers
Notifications	notification_id (PK), user_id (FK), content, is_read, created_time	INT, INT, TEXT, BOOLEAN, DATETIME	Stores push notifications for ride status updates
SOS_Records	sos_id (PK), ride_id (FK), user_id (FK), emergency_location, trigger_time	INT, INT, INT, VARCHAR, DATETIME	Stores safety alert logs for SOS function

Fig. 5. Normalized Relational Database Schema of the Ride-Hailing Platform

All tables have proper primary keys, foreign keys, data types, and constraints. The schema supports efficient queries and ensures data integrity.

D. Implementation Results

The system is fully implemented following object-oriented principles:

- Encapsulation: All fields are private; accessed via public methods.
- Inheritance: Role classes extend the User class.
- Polymorphism: Common interfaces behave differently per role.

Eight modules are correctly implemented and integrated:

1. User Authentication & Management
2. Ride Booking & Matching
3. Real-Time GPS Tracking
4. Payment & Promo Processing
5. Rating & Feedback
6. Push Notification
7. SOS Safety Alert
8. Admin Dashboard

E. Testing Results

All testing levels are completed successfully:

- Unit Testing: All classes and methods pass basic tests.
- Integration Testing: Modules interact correctly.
- System Testing: Full functions run stably.
- Acceptance Testing: All use cases are satisfied.

No critical logical errors are found. The system behaves exactly as designed.

F. Deployment Results

The system is successfully deployable on cloud platforms. Backend APIs, web clients, and mobile APKs run properly. All functions work normally in the deployed environment. The deployment guide is complete and executable.

V. DISCUSSION

A. Requirement Quality and Model Accuracy

The quality of UML models directly depends on the clarity and completeness of requirements. In this project, well-structured, unambiguous, and complete requirements lead to consistent, accurate, and standardized UML diagrams. If requirements were vague, incomplete, or ambiguous, the sequence diagrams and activity diagrams would contain missing messages, wrong orders, or illogical branches.

This observation confirms the core value of requirements engineering: high-quality requirements are the foundation of successful system design. By formalizing requirements before modeling, developers can significantly reduce design errors and rework.

B. Sequential Development and Iterative Improvement

Although this project uses the waterfall model, we recognize that iterative refinement can further improve design quality. For example, reviewing sequence diagrams after finalizing class diagrams helps resolve structural inconsistencies. Adjusting the database schema after verifying interaction flows improves data compatibility.

In practical software engineering, combining the rigorous structure of waterfall with appropriate iterative adjustments produces the most reliable results.

C. Domain Knowledge in System Design

Domain knowledge of ride-hailing operations is critical for creating realistic and practical models. Understanding how driver matching algorithms work, how real-time GPS data is transmitted, how dynamic pricing is calculated, and how payment gateways process transactions helps developers design logical, feasible, and user-friendly systems.

Without sufficient domain knowledge, designers may create oversimplified or unrealistic interactions that do not match real-world scenarios.

D. Model Understandability and Communication Value

The UML models created in this project are highly understandable to both technical developers and non-technical stakeholders. Sequence diagrams and activity diagrams clearly show dynamic behaviors, while class diagrams and database schema present stable structures. Such models improve communication, reduce misunderstanding, and help teams reach consensus quickly.

E. Strengths of the Designed System

The ride-hailing system has several major strengths:

- Complete object-oriented design with standard UML models.
- Full business functions covering real user scenarios.
- Robust and normalized database design.
- Secure authentication and payment processing.
- Comprehensive testing and stable performance.
- Clear documentation and deployable architecture.

F. Limitations of the System

The current prototype also has limitations:

- The driver matching algorithm is basic and can be improved with AI.
- Real-time chat between riders and drivers is not implemented.
- GPS tracking uses simulation instead of actual map API.
- The system uses a single-server architecture without distributed scaling.

G. Future Improvements

Future versions can be enhanced in the following ways:

- Implement intelligent matching based on distance, rating, traffic, and demand.
- Integrate real map APIs such as Google Maps or Amap.
- Add real-time chat and voice call functions.
- Support distributed backend for high concurrency.
- Extend payment methods and provide electronic invoices.
- Develop a data-driven admin dashboard with visual analytics.

VI. THREATS TO VALIDITY

A. Construct Validity

Construct validity refers to whether the models and measurements accurately reflect the concepts they intend to represent. In this project, we strictly follow UML 2.5 specifications and widely accepted OOAD principles. The evaluation criteria—completeness, correctness, standard compliance, understandability, and consistency—are clearly defined based on software engineering standards.

To reduce subjectivity, we refer to classic textbooks, industrial best practices, and academic guidelines. All requirements and models are repeatedly reviewed and verified to ensure alignment.

B. Internal Validity

Internal validity concerns the logical correctness and reliability of the system design and implementation. The development process follows a rigorous waterfall lifecycle, with each phase reviewed before continuation. Test cases cover all use cases and requirements to ensure functional correctness. Database tables follow normalization rules to eliminate redundancy and inconsistency. UML diagrams are validated for syntax and semantic accuracy.

C. External Validity

External validity concerns whether the design can be generalized to other systems and environments. The ride-hailing system is based on real industrial scenarios and universal service functions. Its architecture and modeling method can be reused for food delivery, logistics, car rental, and other on-demand service platforms. Although developed for education, the system adopts practical technologies and structures used in real projects.

D. Scope Limitations

This project is a course prototype and does not aim to support millions of real users. High-concurrency performance and distributed deployment are not fully evaluated. However, within the scope of academic learning and prototype demonstration, the design is complete, reliable, and sufficiently valid.

E. CONCLUSION

This project presents a full lifecycle object-oriented analysis and design of a comprehensive ride-hailing system using standard UML notation and the waterfall software development methodology. The system implements all core functions of modern commercial ride-hailing platforms, including user management, ride booking, real-time GPS tracking, intelligent driver matching, secure payment processing, rating and feedback, scheduled rides, dynamic pricing, push notifications, and SOS emergency support.

The project delivers complete and standardized UML models, including use case diagrams, sequence diagrams, class diagrams, and activity diagrams. A well-designed normalized relational database with seven tables ensures efficient data management. The system is implemented in strict accordance with encapsulation, inheritance, and polymorphism principles. A full testing strategy and cloud deployment plan are also provided to ensure system reliability and executability.

Evaluation results show that the system is complete, correct, compliant with UML standards, and highly understandable. This project demonstrates the complete software development process from requirements gathering to deployment, helping learners master OOAD, UML modeling, requirements engineering, database design, software testing, and software lifecycle management.

In future work, the system can be extended with intelligent algorithms, real map and payment API integration, real-time communication modules, and distributed architecture to support real-world commercial deployment.

REFERENCES

[1] M. Fowler, UML Distilled: A Brief Guide to the Standard Object Modeling Language, 3rd ed. Boston, MA, USA: Addison-Wesley, 2004.

[2] G. Booch, J. Rumbaugh, and I. Jacobson, The Unified Modeling Language User Guide, 2nd ed. Boston, MA, USA: Addison-Wesley, 2005.

[3] I. Sommerville, Software Engineering, 10th ed. Boston, MA, USA: Pearson, 2015.

[4] M. Martin Fowler, Patterns of Enterprise Application Architecture. Boston, MA, USA: Addison-Wesley, 2002.

[5] Uber Technologies Inc., "How Uber Works," 2025. Available: https://www.uber.com/